Activity 1 - Socratic Prompting

I want to build a scalable, data-driven Selenium automation framework.

Do not provide the implementation immediately. Instead, act as a Solution Architect and gather my requirements using the Socratic questioning technique.

Rules:

* Ask exactly one question at a time.

* Each question should provide multiple-choice options (A, B, C, D...) along with a brief explanation of each option.

* Wait for my response before asking the next question.

* Ask only the questions necessary to fully understand my requirements.

* Do not make assumptions unless I explicitly ask for recommendations.

Once all the requirements are gathered:

1. Summarize my chosen requirements.

2. Explain the architecture and design decisions.

3. Provide the recommended project structure.

4. Implement the complete framework step by step with well-commented Java code.

5. Explain why each design choice was made and discuss possible alternatives.

6. Follow industry best practices for Selenium, TestNG, Maven, Page Object Model, and data-driven testing.

Response from ChatGPT:

Perfect. I'll leave **test execution state/shared-context management as an explicit architecture decision for the detailed design phase** rather than assuming a model.

Solution Architecture — Scalable Data-Driven Automation Framework

Based strictly on your answers, the target is a **Java-based, scalable automation platform** with Selenium UI automation and independent API automation capabilities.

1. Confirmed Requirements

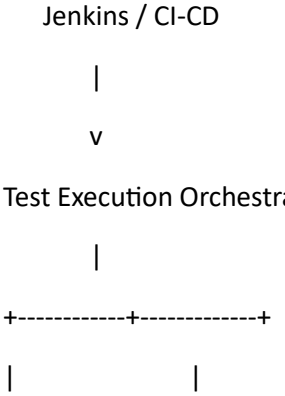
Area	Confirmed requirement
Primary scope	Web UI initially, extensible architecture
Applications	Multiple applications with different authentication mechanisms
UI technology	Selenium
Test framework	TestNG
Design pattern	POM + component objects + business/application abstractions
Data source	PostgreSQL
Test data	Dynamically generated
Data cleanup	Test-owned data with automatic cleanup
Environment	Configuration + runtime override
Authentication	Hybrid authentication architecture
Parallel execution	Test + browser level
WebDriver lifecycle	Configurable
Browser configuration	Configuration + runtime/CI/CD overrides
Browsers	Chrome + Edge
Reporting	Allure
Logging	Configurable logging
Failure diagnostics	Screenshot + page source + browser console logs + metadata
Test organization	Hybrid TestNG groups/suites/annotations/configuration
Synchronization	Layered explicit-wait strategy

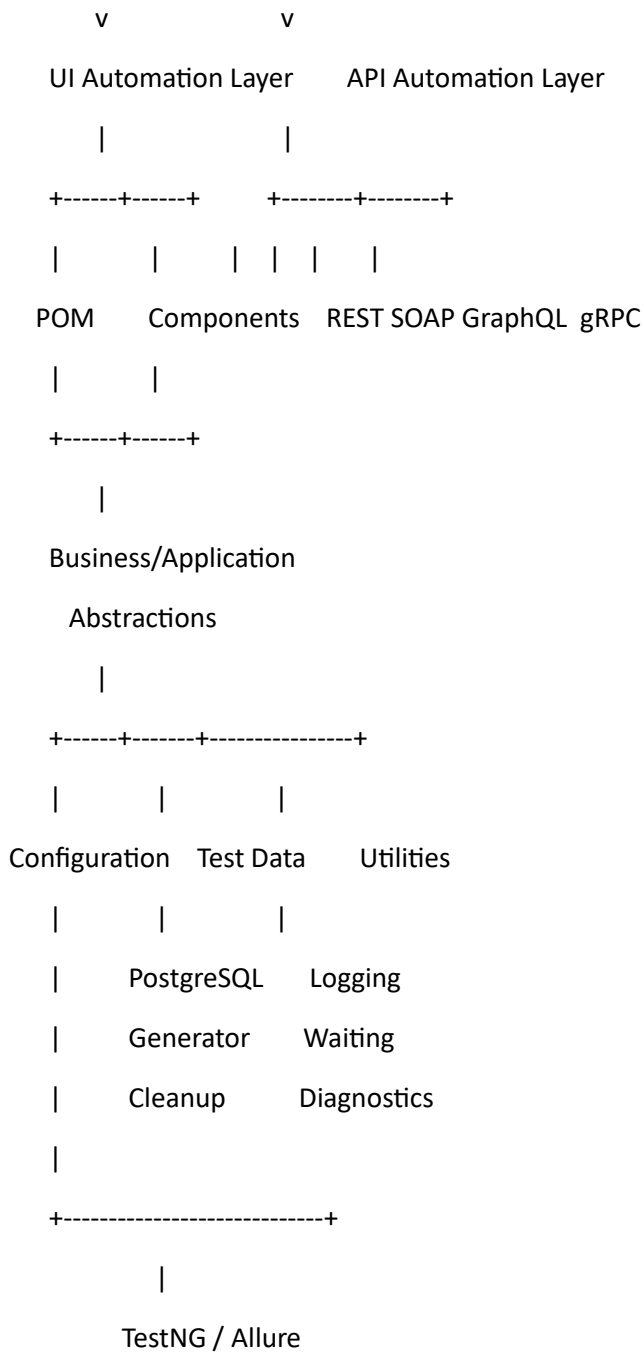
Area	Confirmed requirement
Retry	Configurable + failure-aware
Build	Maven with enterprise structure
Java	Latest stable version compatible with project dependencies/environment
CI/CD	Jenkins
CI/CD triggering	Manual + scheduled + code-change
Browser drivers	Managed by CI/CD/execution environment
Test selection	External test-management abstraction
Test-management product	Not decided
Selenium Grid	Not in initial scope
API automation	Required
API technologies	REST + SOAP + GraphQL + gRPC
API/UI relationship	Independent initially
Maven/repository structure	Organization-specific
Test independence	Configurable
Environment behavior	Hybrid configuration + implementation
Shared execution context	Not decided

---

2. High-Level Architecture

The framework should be designed as **layers rather than one large Selenium framework**.





The important architectural principle is **separation of concerns**.

Tests should describe **what is being validated**, while framework layers handle **how it is executed**.

---

3. UI Automation Architecture

The UI side should follow:

Test

↓

Business/Application Layer

↓

Page Objects / Components

↓

WebDriver Abstraction

↓

Selenium

↓

Browser

Page Objects

Page classes should contain:

- Locators
- Page-specific actions
- Page-specific validations
- Navigation behavior

They should **not** contain:

- Database code
- Test data generation
- Jenkins logic
- Reporting implementation
- Environment-specific credentials
- Test orchestration

Component Objects

Reusable UI components should be extracted where appropriate:

Header

NavigationMenu

DatePicker

Table

Modal

ToastMessage

Dropdown

This prevents large page classes from becoming difficult to maintain.

---

4. Business/Application Layer

Because you selected **POM + components + business/application abstractions**, tests should preferably operate at a business level.

For example:

```
Test
 |
 +-- CreateCustomerWorkflow
     |
     +-- CustomerPage
     +-- AddressComponent
     +-- ConfirmationComponent
```

Rather than putting every UI interaction directly inside the test.

This gives you an important scalability advantage: **UI implementation changes can often be isolated inside the page/component layer without rewriting test scenarios.**

---

5. API Architecture

API automation should be implemented as a **separate capability** from UI automation initially.

```
API Tests
   |
   v
API Abstraction
   |
   +------------------------------+
   |       |       |       |
  REST    SOAP   GraphQL   gRPC
```

Each API technology should have its own implementation layer.

The architecture should avoid forcing REST, SOAP, GraphQL and gRPC into one artificial implementation.

Instead, provide common framework services such as:

- Configuration
- Authentication support

- Logging
- Reporting
- Test data
- Validation utilities
- Error handling

while keeping protocol-specific implementation separate.

---

6. Data-Driven Architecture

Your requirement is:

PostgreSQL + dynamically generated test data + automatic test-owned cleanup.

A suitable conceptual flow is:

Test

|

v

Test Data Factory

|

v

Dynamic Data Generator

|

v

PostgreSQL

|

v

Application Test

|

v

Data Ownership Tracker

|

v

Cleanup

Important design principle

Every piece of generated test data should have an identifiable **test ownership relationship**.

For example:

Execution ID

|

+-- Test ID

|

+-- Customer ID

+-- Order ID

+-- Account ID

This becomes especially important once parallel execution is introduced.

You explicitly left the **parallel test-data isolation strategy undecided**, so the exact implementation should be chosen during detailed architecture design rather than assumed now.

---

7. Configuration Architecture

You selected:

Configuration + runtime override.

Therefore:

Default Configuration

|

v

Environment Configuration

|

v

Runtime Parameters

|

v

Effective Configuration

For example:

Environment

Browser

Headless

Timeout

Execution mode

Test category

API endpoint

UI endpoint

can have defaults while Jenkins can override them.

Secrets

You selected **CI/CD secret management**.

Therefore sensitive credentials should **not be stored in source-controlled configuration files**.

Conceptually:

Framework Configuration

+

Jenkins Parameters

+

Jenkins Credentials

|

v

Effective Runtime Configuration

---

8. Authentication Architecture

You selected a **hybrid authentication approach**.

Therefore the framework should have an authentication abstraction capable of supporting different strategies.

Conceptually:

Authentication Manager

|

+-- Username/Password

|

+-- SSO

|

+-- OAuth

|

+-- Application-specific authentication

|

+-- External/pre-authenticated session

The exact authentication mechanisms should be implemented based on the applications rather than assumed in the framework.

---

9. WebDriver Architecture

You selected:

- Configurable lifecycle
- Test + browser parallelism
- Browser drivers managed by CI/CD
- Chrome + Edge

Therefore WebDriver management should be isolated behind a framework abstraction.

Test

|

v

Driver Manager / Factory

|

v

Execution Configuration

|

v

WebDriver

|

+---- Chrome

|

+---- Edge

The framework should **not download or manage browser drivers**.

The execution environment is responsible for driver availability.

You also selected configurable behavior when browser/driver initialization fails:

Driver initialization failure

|

v

Configured strategy

/ | \

Fail Skip Retry

---

10. Parallel Execution

Your requirement is:

Test + browser-level parallelism.

Therefore the framework must be designed with concurrency in mind from the beginning.

Conceptually:

TestNG

|

+----------+----------+

| | |

Thread 1 Thread 2 Thread 3

| | |

Chrome Edge Chrome

| | |

Test A Test B Test C

Critical areas requiring isolation:

- WebDriver
- Test data
- Execution metadata
- API clients where stateful
- Temporary files
- Reporting context

You explicitly selected **“not decided yet”** for shared execution state and parallel test-data isolation. Those should remain open architectural decisions.

---

11. Synchronization

Your selected strategy is a **layered approach**.

The framework should therefore establish:

Standard explicit waits

↓

Centralized wait utilities

↓

Configurable timeout/polling

↓

Specialized waits when required

Fixed Thread.sleep() calls should not be the normal synchronization mechanism.

---

12. Test Organization

You selected:

Configurable test categories.

And external test-management integration is also required, but the specific product is undecided.

Therefore the framework should avoid hard-coding something like:

Xray

Zephyr

TestRail

into the test architecture.

Instead:

Test Management Abstraction

|

+-- Future Adapter A

+-- Future Adapter B

+-- Future Adapter C

This keeps the framework independent of the eventual test-management platform.

---

13. Retry Architecture

You selected:

Configurable + failure-aware retry.

Therefore retries should **not simply rerun every failed test blindly**.

Conceptually:

Test Failure

|

v

Failure Classification

|

+---+----------+

| |

Transient Functional

| |

Retry Don't retry

Retry count should be configurable.

This prevents retries from hiding genuine application defects.

---

14. Failure Diagnostics

For failures, the framework should collect:

- Screenshot
- Page source
- Browser console logs
- URL
- Browser
- Environment
- Test name
- Timestamp
- Relevant execution metadata

Conceptually:

Failure

|

+-- Screenshot

+-- Page Source

+-- Console Logs

+-- Execution Metadata

|

v

Allure

---

15. Reporting

You selected **Allure**.

Allure should become the primary human-readable reporting layer.

The framework should expose reporting through framework-level abstractions where practical, rather than spreading reporting implementation throughout every page object.

For example:

Test

|

+-- Business Action

| |

| +-- Allure step

|

+-- Validation

   |

   +-- Allure result

---

16. Logging

You selected **configurable logging**.

The framework should therefore support configurable levels such as:

ERROR

WARN

INFO

DEBUG

The logging implementation should be centralized rather than creating independent logging approaches inside every framework component.

---

17. Artifact Management

You selected:

CI/CD artifacts + configurable retention.

Therefore Jenkins should publish:

Allure results/report

Screenshots

Page source

Console logs

Framework logs

Other diagnostic artifacts

with retention controlled according to CI/CD configuration.

---

18. Jenkins Architecture

Your CI/CD platform is **Jenkins**.

The framework should therefore expose a predictable command-line execution interface.

Conceptually:

```
Git
 |
 v
Jenkins
 |
 +-- Build
 |
 +-- Resolve dependencies
 |
 +-- Select environment
 |
 +-- Select test category
 |
 +-- Select browser
```

|

+-- Select parallelism

|

+-- Execute TestNG

|

+-- Generate Allure results

|

+-- Publish artifacts

You selected three trigger mechanisms:

- Manual
- Scheduled
- Code-change triggered

So the Jenkins pipeline should support all three.

---

19. Browser Matrix

Your confirmed browser scope is:

Chrome + Edge

You haven't specified browser versions, so I will **not invent a browser-version matrix**.

Browser/platform versions should remain configuration supplied by the execution environment when you define them.

---

20. Maven & Java

The framework should use:

Maven with your organization's existing enterprise structure.

You also requested the **latest Java version**.

Rather than claiming a specific version without validating your organization's compatibility requirements, the architecture requirement is:

Use the latest stable Java release that is compatible with the selected Selenium/TestNG/API dependencies, Maven plugins, Jenkins runtime, and organizational build infrastructure.

The exact Java version should be locked during implementation/build setup.

---

21. What We Have Deliberately NOT Assumed

These remain open because you explicitly did not provide the information:

Parallel test-data isolation

Not yet decided.

Execution context/state model

Not yet decided.

Test-management platform

Not yet decided.

Enterprise Maven/repository structure

Must follow your organization's existing standard.

Exact API libraries

The API protocols are known:

- REST
- SOAP
- GraphQL
- gRPC

But the specific Java libraries/frameworks have **not** been selected.

Browser versions

Only Chrome and Edge are confirmed.

Selenium Grid/cloud

Not part of the initial scope.

---

22. Architectural Principles

The framework should follow these principles:

1. **Tests should contain business intent, not low-level Selenium implementation.**
2. **Page objects should contain UI behavior, not test orchestration.**
3. **Reusable UI components should be extracted from complex pages.**
4. **API protocols should remain independently implemented.**
5. **Configuration should be externalized.**
6. **Secrets should never be committed to source control.**
7. **Test data should be dynamically generated and owned by its test.**
8. **Parallel execution must be considered from the framework foundation.**

9. **Framework utilities should be centralized and reusable.**
10. **External systems should be accessed through abstractions/adapters where the actual product hasn't been selected.**
11. **CI/CD should execute the framework rather than become part of test logic.**
12. **Framework behavior should be configurable rather than hard-coded.**

---

23. Target Logical Structure

Without imposing your organization's exact repository structure, the logical architecture would resemble:

Automation Framework

|

├— Core

| ├— Configuration

| ├— Logging

| ├— Reporting

| ├— Exceptions

| ├— Utilities

| ├— Waits

| └— Execution Context

|

├— UI

| ├— Driver

| ├— Pages

| ├— Components

| ├— Business Workflows

| └— UI Utilities

|

├— API

| ├— REST

| ├— SOAP

| ├— GraphQL

```
|   └── gRPC
|
├── Test Data
|   ├── PostgreSQL
|   ├── Data Generators
|   ├── Data Ownership
|   └── Cleanup
|
├── Authentication
|   ├── Common Abstraction
|   ├── Application Strategies
|   └── External Session Support
|
├── Tests
|   ├── UI Tests
|   └── API Tests
|
└── CI/CD
    └── Jenkins integration
```

This is a **logical architecture**, not a prescription for your organization's physical Maven/repository layout.

Final Architectural Position

The framework you're describing is no longer just a Selenium POM project. It is better treated as a **scalable automation platform** with:

Selenium UI + TestNG + POM/components/business abstractions + PostgreSQL-driven dynamic data + REST/SOAP/GraphQL/gRPC automation + Allure + configurable execution + Jenkins CI/CD.

The most important architectural decisions still intentionally left open are **parallel test-data isolation, execution-context/state management, exact API libraries, test-management integration, enterprise repository structure, and browser versions**.

I have not filled those gaps with assumptions.

\\\